# C++ Objects & Ownership

Copy/move policy, borrowed lifetimes, cleanup and invalidation, with explicit C++17 and host-check scope.

**Debug build:**
`-std=c++17 -Wall -Wextra -g -fsanitize=address,undefined`
**Also try:** `-O2 · -fsanitize=thread`

bug example (see captured check) · fixed example (checks have bounded scope) · ▶ = what gcc (GCC) 16.2.1 20260810 / its sanitizers actually reported · 11 entries

## CPP-OBJ-01 Rule of 3/5 violation (shallow copy)

*SR-404 · Climate-control buffer*

× BUGGY

```
struct Buffer {
    char* data;
    explicit Buffer(std::size_t n) : data(new char[n]) {}
    ~Buffer() { delete[] data; }
};   // copy constructor/assignment are still the implicit ones
// Buffer a(16); Buffer b = a; // implicit copies alias one allocation
```

▶ `ASan: attempting double-free`

**What goes wrong:** In this Buffer, implicit copying copies the pointer value, not the allocated characters. Copying a successfully allocated buffer creates two handles whose destructors attempt to delete the same allocation. Raw pointers are not inherently owners; this example makes data an owner by its destructor.

> FIX

```
struct Buffer {                          // Rule of Zero
    std::vector<char> data;
    explicit Buffer(std::size_t n) : data(n) {}
};   // member operations implement this character-value policy
```

**Avoid it:** Author guidance: prefer resource-managing members when they express the intended value/ownership policy. Here vector gives independent character storage on copy. The Rule of Zero/Five is design guidance, not an ISO requirement to hand-write five functions; review defaulted, deleted and custom operations together.

Sources: C++17 draft 15.8.1 [class.copy.ctor]; 15.8.2 [class.copy.assign]; 26.3.11 [vector]; C++ Core Guidelines (guidance) C.20; C.21

## CPP-OBJ-02 Object slicing

*SR-404 · Sensor polling*

× BUGGY

```
struct Sensor {
    virtual ~Sensor() = default;
    virtual int read() const { return 0; }
};
struct RpmSensor : Sensor {
    int read() const override { return 3000; }
};
int poll(Sensor s) { return s.read(); }  // slices: base part only
```

▶ `prints: poll=0`

**What goes wrong:** Constructing the by-value Sensor parameter from RpmSensor creates a separate Sensor object. Calling read on that parameter uses Sensor's implementation; it does not change the original derived object. Likewise, constructing vector<Sensor> elements from derived values cannot preserve their dynamic type.

> FIX

```
// Sensor and RpmSensor unchanged
int poll(const Sensor& s) { return s.read(); }  // dispatches
```

**Avoid it:** Author guidance: use references/pointers when dynamic dispatch to the original live object is intended. Owning unique_ptr elements are one storage policy, not the only one. Reference/pointer lifetimes still matter; the supplied temporary remains alive through the poll call.

Sources: C++17 draft 15.8.1 [class.copy.ctor]; 13.3 [class.virtual]; 15.2 [class.temporary]

## CPP-OBJ-03 Raw new/delete leaks on exceptions

*SR-404 · Fuel-map tuning*

× BUGGY

```
void tune_fuel_map() {
    auto* map = new FuelMap();
    calibrate(*map);      // throws → delete never runs
    delete map;
}
```

▶ `LeakSanitizer: detected memory leaks`

**What goes wrong:** In the supplied fixture, calibrate throws after allocation and an outer handler catches it. The raw pointer has no cleanup operation on that unwinding path. With unique_ptr, destruction during this unwinding releases the object. Not every exception/early return between allocation and delete necessarily leaks.

> FIX

```
void tune_fuel_map() {
    auto map = std::make_unique<FuelMap>();
    calibrate(*map);  // outer catch unwinding releases the owner
}
```

**Avoid it:** Author guidance: express owned resources through RAII members or handles, with the correct deleter. The supplied catch is part of this cleanup demonstration; terminating paths do not universally promise stack unwinding. Clang documents platform/configuration limits for leak detection; the host receipt covers this particular run.

Sources: C++17 draft 18.2 [except.ctor]; 18.5.1 [except.terminate]; 23.11.1.2.2 [unique.ptr.single.dtor]; Clang docs Memory leak detection; Limitations; C++ Core Guidelines (guidance) R.11; R.20; E.6

## CPP-OBJ-04 Iterator invalidation

*SR-404 · Fault-code list*

× BUGGY

```
std::vector<int> faults{1, 2, 3};
while (faults.size() < faults.capacity())
faults.push_back(0);
for (int f : faults)
    if (f == 2)
        faults.push_back(99);  // exceeds capacity: invalidates these iterators
```

▶ `ASan: heap-use-after-free`

**What goes wrong:** Range-for caches begin/end. N4659 preserves pre-insertion handles without reallocation; N4861 explicitly invalidates the old end even without reallocation. This bad fixture fills spare capacity first, so append requires reallocation and invalidates element iterators under both drafts. Erase/reserve have distinct rules.

> FIX

```
std::vector<int> faults{1, 2, 3};
const std::size_t n = faults.size();
for (std::size_t i = 0; i < n; ++i)  // original range; append only
    if (faults[i] == 2)
        faults.push_back(99);
```

**Avoid it:** Author guidance: this append-only fix visits the original n elements by index and keeps no element reference across append. It does not cover erasing/shrinking or every container. Reallocation invalidates element handles; reserve within current capacity does not. Sanitizer/debug checks have executed-path and library scope.

Sources: C++17 draft 9.5.4 [stmt.ranged]; 26.3.11.3 [vector.capacity]; 26.3.11.5 [vector.modifiers]; Clang docs Introduction; Limitations; C++20 draft N4861 [vector.modifiers]: past-the-end wording

## CPP-OBJ-05 Dangling std::string_view

*SR-404 · Radio station display*

× BUGGY
```
std::string_view station_name(int mhz) {
    std::string s = "KRAD " + std::to_string(mhz) + " FM
STEREO";
    return s;              // view into a string destroyed on
return
}
```

▸ ASan: heap-use-after-free

**What goes wrong:** This string_view refers to characters owned by the local string, whose lifetime ends at return. The view does not extend that lifetime. Invalidation can also happen before owner destruction, for example on storage reallocation; views into static backing storage have a different lifetime contract.

> FIX
```
std::string station_name(int mhz) {          // return the
owner
    return "KRAD " + std::to_string(mhz) + " FM STEREO";
}
```

**Avoid it:** Author guidance: return an owning string here, or document and maintain the lifetime of borrowed storage. A view may be a valid return when its backing data outlives use. Compiler/sanitizer diagnostics vary by version, storage choice and path; the supplied host capture is not a promise that every dangling view is detected.

Sources: C++17 draft 24.4.2 [string.view.template]; 6.8 [basic.life]; 24.3.2 [basic.string]; Clang docs Introduction; Limitations

## CPP-OBJ-06 Missing virtual destructor

*SR-404 · ECU base class*

× BUGGY
```
struct Ecu {
    virtual void tick() {}
};                                  // destructor is NOT
virtual
struct Radio : Ecu {
    std::string station = "KRAD 101.5 FM STEREO";
};
void scrap(Ecu* e) { delete e; }  // nonnull Radio*:
undefined in C++17
```

▸ gcc -Wdelete-non-virtual-dtor: deleting object of polymorphic class ty…

**What goes wrong:** In this C++17 example, deleting a nonnull Radio through Ecu* without a virtual base destructor is undefined. That rule does not specify which destructors physically run or require a particular sanitizer report. Deleting a null pointer is a separate permitted case.

> FIX
```
struct Ecu {
    virtual ~Ecu() = default;        // derived destructors
now run
    virtual void tick() {}
};
struct Radio : Ecu {
    std::string station = "KRAD 101.5 FM STEREO";
};
void scrap(Ecu* e) { delete e; }  // requires ownership and a
valid live object or nullptr
```

**Avoid it:** Author guidance: a base intended for polymorphic deletion can expose a public virtual destructor; protected nonvirtual destruction is another policy when base deletion must be forbidden. GCC14 documents selected deletion/class warnings. Owning smart pointers still need a valid deletion contract.

Sources: C++17 draft 8.3.5 [expr.delete]p3; 15.4 [class.dtor]; GCC14 docs -Wdelete-non-virtual-dtor; -Wnon-virtual-dtor; C++ Core Guidelines (guidance) C.35; GCC14 C++ options -Wdelete-non-virtual-dtor; -Wnon-virtual-dtor

## CPP-OBJ-07 Throwing from a destructor

*SR-404 · Trip log writer*

× BUGGY
```
struct TripLog {
    bool flush() noexcept { return false; } // simulated
failure
    ~TripLog() {
        if (!flush())
            throw std::runtime_error("flush");  // →
std::terminate
    }
};
```

▸ gcc -Wterminate: 'throw' will always call 'terminate'

**What goes wrong:** This TripLog destructor has an implicit nonthrowing specification: its subobjects do not have potentially throwing destructors. Letting its exception escape calls terminate. Other destructors can have an implicit potentially throwing specification. An escaping destructor exception during exception unwinding also terminates.

> FIX
```
struct TripLog {
    bool flush() noexcept { return false; } // simulated
failure
    [[nodiscard]] bool close() noexcept {
        close_attempted_ = true; return flush();
    }
    ~TripLog() noexcept {
        if (!close_attempted_)
            (void)flush();  // simulated failure ignored; no
exception
    }
private:
    bool close_attempted_ = false;
};
```

Sources: C++17 draft 18.4 [except.spec]p8; 18.5.1 [except.terminate]; C++ Core Guidelines (guidance) E.16; GCC14 C++ options -Wterminate

**Avoid it:** Author guidance: report expected failure through an explicit operation and keep cleanup nonthrowing. Here flush is a simulated failure with no real I/O; close reports false, marks an attempt and prevents a destructor retry. It is not a durable file-commit API. GCC14 documents -Wterminate for selected escaping throws.

## CPP-OBJ-08 Assuming moved-from contents are unchanged

*SR-404 · RPM sample history*

× BUGGY
```
std::vector<int> samples{3000, 3100, 3200};
store(std::move(samples));
if (samples.size() != 3) { // unjustified original-size
assumption
    std::cout << "original-size assumption failed\n";
    return 1;
}
```

▸ prints: original-size assumption failed

**What goes wrong:** std::move is a cast; the call's by-value construction performs the move. For standard-library types, absent a more specific contract, the source is valid with unspecified state. Calling vector::size after this move is valid. Assuming the old three elements remain is the error illustrated by the bad branch.

> FIX
```
std::vector<int> samples{3000, 3100, 3200};
std::cout << "before=" << samples.size() << '\n';
store(std::move(samples));
samples.clear(); // known empty state; queries were already
valid
samples.push_back(3300);
std::cout << "reused=" << samples.front() << '\n';
```

**Avoid it:** Author guidance: check each moved-from operation's preconditions. Queries such as size/empty are valid; front needs a nonempty vector. clear establishes an empty state before the shown reuse, but assignment/clear is not required before every operation. User-defined move behavior needs its own contract.

Sources: C++17 draft 23.2.5 [forward]; 20.5.5.15 [lib.types.movedfrom]; 20.3.25 [defns.valid]; 26.3.11 [vector]

## CPP-OBJ-09 new[] paired with delete

*SR-404 · Temperature table*

× BUGGY
```
int* temps = new int[8]();
delete temps;              // must be delete[]: UB
```

▸ gcc -Wmismatched-new-delete: 'void operator delete(void*, std::size_t)…

**What goes wrong:** For this pointer from array new, a single-object delete violates C++17's deallocation contract. The language rule does not depend on an assumed allocation cookie or allocator layout. The other allocation/deallocation families also require their specified matching ownership and release operations.

> FIX
```
std::vector<int> temps(8);              // owns and sizes
itself
auto raw = std::make_unique<int[]>(8);  // if you need a
plain array
```

**Avoid it:** Author guidance: choose an array-owning vector, fixed-size array, or unique_ptr<T[]> where appropriate. The shown vector and make_unique array contain eight value-initialized ints. GCC14 documents -Wmismatched-new-delete in -Wall; sanitizer diagnostics remain tool/configuration-specific.

Sources: C++17 draft 8.3.5 [expr.delete]; 23.11.1.4 [unique.ptr.create]; 26.3.11.2 [vector.cons]; GCC14 docs -Wmismatched-new-delete; Clang docs Introduction; Usage; Limitations; GCC14 C++ options -Wmismatched-new-delete

## CPP-OBJ-10 Reference to a temporary through a function

*SR-404 · Cabin heater label*

× BUGGY

```
const std::string& label(const std::string& s) { return s; }
void show() {
    const std::string& t = label("CABIN HEATER MAX");  // temp!
    std::cout << t << '\n';  // dead: it died at the ';' above
}
```

▸ `gcc -Wdangling-reference: possibly dangling reference to a temporary`

**What goes wrong:** Here the converted string temporary bound to label's reference parameter lasts through the initializer's full-expression. Returning the reference does not extend it to t's lifetime, so the later stream access is invalid. Copying the referred string into an owning result before the full-expression ends is valid.

> FIX

```
const std::string& label(const std::string& s) { return s; }
void show() {
    std::string t = label("CABIN HEATER MAX");  // owns a copy
    std::cout << t << '\n';
}
```

**Avoid it:** Author guidance: use a value result when a borrowed reference may escape its backing lifetime. A function may safely return a reference to suitably long-lived input; min/max on temporaries are problematic when their returned reference is retained past the full-expression, not every immediate value use. Tool findings are bounded diagnostics.

Sources: C++17 draft 15.2 [class.temporary]p6; 28.7.8 [alg.min.max]; Clang docs Introduction; Limitations; GCC14 C++ options -Wdangling-reference

## CPP-OBJ-11 shared_ptr reference cycle

*SR-404 · Paired cabin ECUs*

× BUGGY

```
struct Node {
    std::string name;
    std::shared_ptr<Node> peer;      // strong in both directions
};
void pair_ecus() {
    auto a = std::make_shared<Node>(), b = std::make_shared<Node>();
    a->peer = b;
    b->peer = a;  // strong cycle remains after these local owners leave
}
```

▸ `LeakSanitizer: detected memory leaks`

**What goes wrong:** If pair_ecus completes normally with both strong peer links, releasing the local owners leaves each node owned by the other. This function does not break that cycle, so its nodes remain allocated. Reference cycles can be deliberately broken; forever and typical-frequency claims require separate context.

> FIX

```
struct Node {
    std::string name;
    std::weak_ptr<Node> peer;        // observes, doesn't own
};
void pair_ecus() {
    auto a = std::make_shared<Node>(), b = std::make_shared<Node>();
    a->peer = b;
    b->peer = a;                     // no strong peer cycle
}
```

**Avoid it:** Author guidance: design explicit ownership and observation relationships. In this fixture weak peer links avoid a strong cycle, so both nodes are destroyed when the local owners leave scope. For later observation, lock a weak_ptr and check the returned shared_ptr before access; lock alone is not a liveness guarantee.

Sources: C++17 draft 23.11.2.2.2 [util.smartptr.shared.dest]; 23.11.2.3 [util.smartptr.weak]; 23.11.2.3.5 [util.smartptr.weak.obs]; Clang docs Memory leak detection; Limitations

Bug examples were compiled; runnable cases were executed on 2026-10-27T00:32:15+0000 with gcc (GCC) 16.2.1 20260810 (C++17). Full sources, including the hidden test drivers: `cheatsheets/src/snippets/<ID>/`. Mistake IDs match `docs/mistakes_catalog.json`, so in-game bugs can link here. Rendering does not repeat compilation. Public drafts are identified in the references; passing checks do not prove all-input correctness. Author correction draft: independent content review remains pending.